

# CSE374: Machine Learning Classification Project

## Final Project Report

**Dataset: Credit Card Default (Taiwan)**

Models: Logistic Regression | Decision Tree | k-Nearest Neighbours

Ain Shams University — Computer Science Engineering

Course: CSE374 — Machine Learning and Pattern Recognition

| Team Member            | Role                                | Student ID |
|------------------------|-------------------------------------|------------|
| Jana Sameh Hamed       | Role 1 — Data & Preprocessing Lead  | 2301019    |
| Marwa Mohamed Hassan   | Role 2 — EDA & Visualization Lead   | 2301138    |
| Khaled Mohamed Hussein | Role 3 — Modeling & Evaluation Lead | 2301037    |

# Section 1: Problem Framing & Data Understanding

---

## 1.1 Problem Description

Banks and financial institutions need to assess the credit risk of their customers before extending or renewing credit lines. The core question is: given a customer's demographic profile, credit history, and recent payment behaviour, can we predict whether they will default on their next monthly payment? Accurately identifying likely defaulters allows the bank to take preventive action before a loss occurs. This study applies supervised machine learning classification to the Credit Card Default dataset from Taiwan to build and compare predictive models for this task.

## 1.2 Target Variable

The target variable is `default.payment.next.month` : a binary indicator where 1 means the customer defaulted and 0 means they did not. In the notebook it was renamed to "default" for readability. The prediction task is therefore a binary classification problem with two possible outcomes per customer.

## 1.3 Classification Type

This is a binary classification problem. The model outputs one of two outcomes: default (1) or no default (0). Binary classification enables the use of standard evaluation metrics including Accuracy, F1-Score, Confusion Matrix, and ROC/AUC, all of which are reported in this study.

## 1.4 Anticipated Challenges

| Challenge           | Description                                                           | Mitigation Strategy                                                             |
|---------------------|-----------------------------------------------------------------------|---------------------------------------------------------------------------------|
| Class Imbalance     | ~22% default rate                                                     | <code>class_weight="balanced"</code> ; report F1 and ROC-AUC as primary metrics |
| Feature Correlation | Six monthly <code>BILL_AMT</code> columns highly correlated (0.7-0.9) | Noted as LR limitation; discussed in error analysis                             |
| Mixed Feature Types | Categorical integers (SEX, EDUCATION, MARRIAGE) + continuous amounts  | Excluded categoricals from <code>StandardScaler</code>                          |
| Noisy Categoricals  | Undocumented values in EDUCATION (0,5,6) and MARRIAGE (0)             | Merged into existing "other" categories during cleaning                         |

## Section 2: Data Acquisition & Documentation

---

### 2.1 Dataset Source

It was originally collected by researchers at National Cheng Kung University, Taiwan, and covers credit card clients from April to September 2005. It has been widely used in academic literature on credit risk modelling.

<https://archive.ics.uci.edu/dataset/350/default+of+credit+card+clients>

### 2.2 Dataset Overview

| Property        | Detail                                               |
|-----------------|------------------------------------------------------|
| Total Instances | 30,000                                               |
| Input Features  | 23                                                   |
| Target Column   | default.payment.next.month (renamed to "default")    |
| File Format     | XLS (Excel 97-2003 — requires xlrd library)          |
| Missing Values  | None (confirmed via df.info())                       |
| Class Balance   | 22.1% default (class 1) / 77.9% no default (class 0) |
| Time Period     | April–September 2005 (6 months of payment history)   |

## Section 4: Data Cleaning & Preprocessing

---

All preprocessing was performed in a deliberate sequence: inspection → cleaning → splitting → scaling. This ordering is critical :scaling is applied after splitting to prevent data leakage from validation/test statistics into the training scaler.

### 4.1 Column Renaming and ID Removal

The target column was renamed from "default payment next month" to "default". The ID column was dropped as it is a sequential row number with no relationship to credit behaviour ,including it would cause data leakage by allowing models to associate ID ranges with default rates specific to the training split.

### 4.2 Categorical Value Cleaning

| Column    | Undocumented Values | Count           | Action                           |
|-----------|---------------------|-----------------|----------------------------------|
| EDUCATION | 0, 5, 6             | 468 rows (1.6%) | Merged into category 4 ("other") |
| MARRIAGE  | 0                   | 54 rows (0.18%) | Merged into category 3 ("other") |
| SEX       | None                | 0 rows          | No action required               |

### 4.3 Train / Validation / Test Split

| Split            | Rows   | Default Rate | Purpose                                    |
|------------------|--------|--------------|--------------------------------------------|
| Training (70%)   | 21,012 | 22.1%        | Model learns patterns from this data       |
| Validation (15%) | 4,488  | 22.1%        | Hyperparameter tuning and model comparison |
| Test (15%)       | 4,500  | 22.1%        | Final evaluation                           |

Stratified splitting (stratify=y) was applied in both split calls to preserve the 22.1% default rate across all three sets. The consistent default rate across splits was verified after splitting.

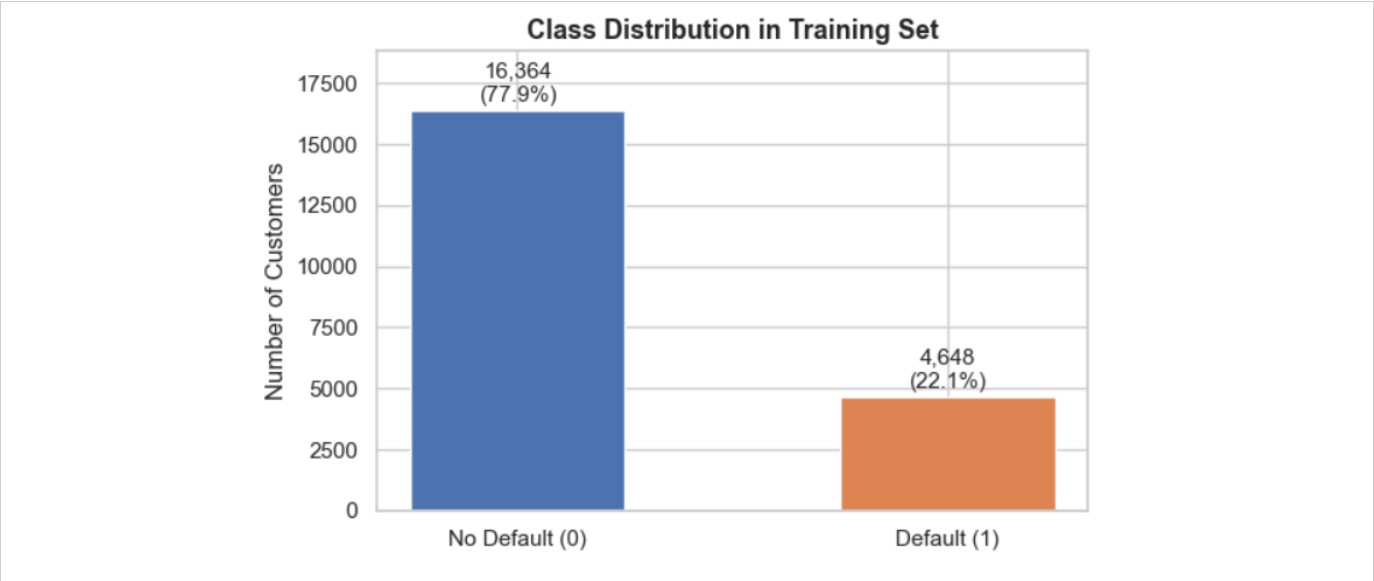
### 4.4 Feature Scaling

StandardScaler was applied to 20 of 23 features ( all columns except SEX, EDUCATION, and MARRIAGE). The three categorical columns were excluded because their integer codes represent categorical labels, not continuous quantities. The scaler was fit exclusively on the training set and then applied (transform only) to validation and test sets. Fitting on the full dataset would constitute data leakage.

## Section 5: Exploratory Data Analysis (EDA)

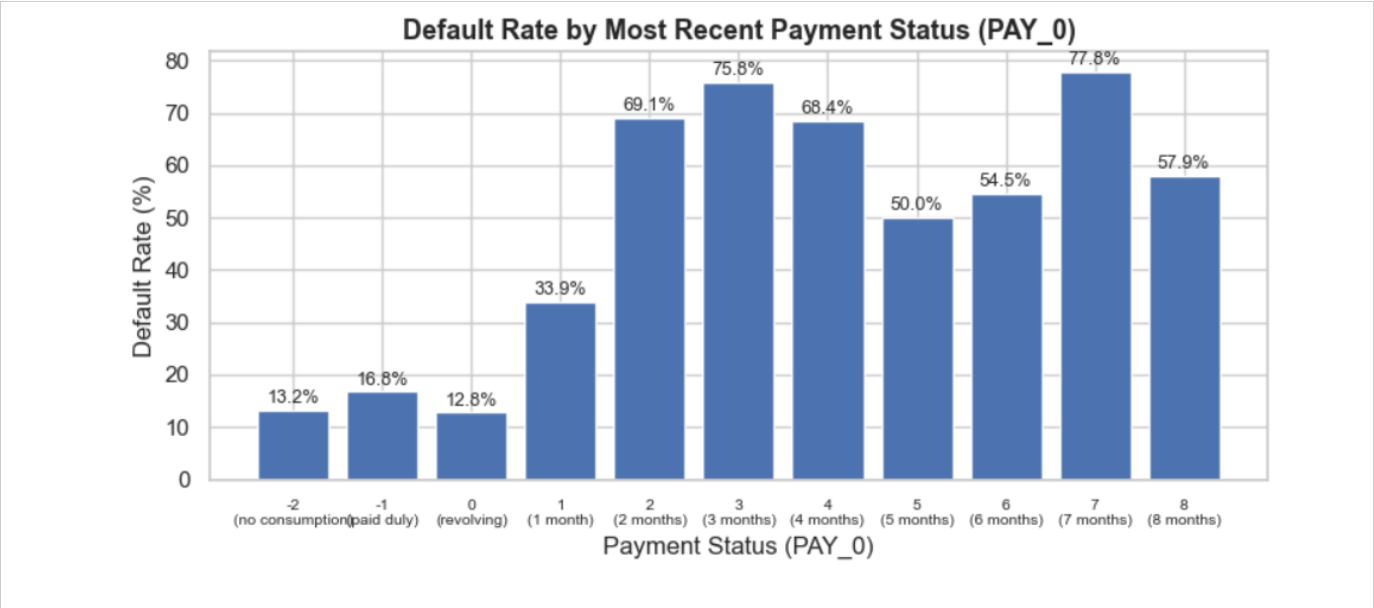
Four visualisations were produced, each chosen to answer a specific question and directly inform modelling decisions. Plots were produced using matplotlib and seaborn with a consistent whitegrid style.

### 5.1 Plot 1 — Class Distribution



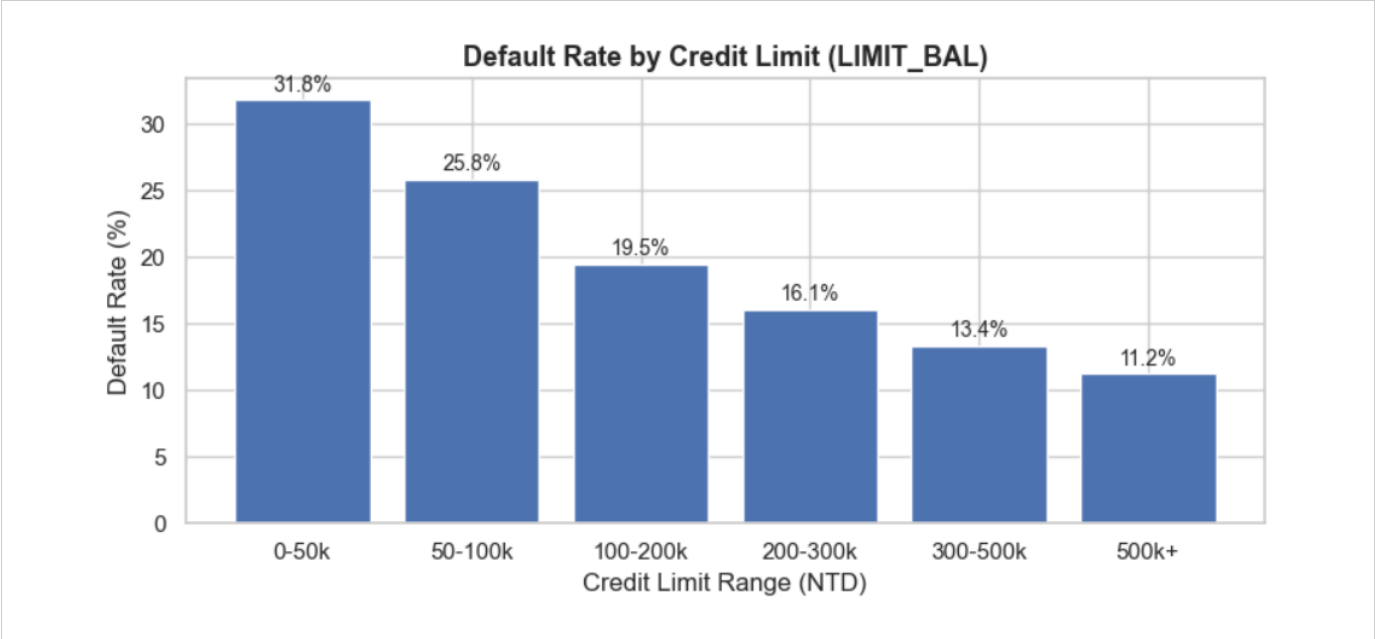
The training set contains 16,364 non-default and 4,648 default cases (22.1% default rate). A model predicting "no default" for every customer would achieve 77.9% accuracy while being completely useless. This finding directly justifies using `class_weight="balanced"` in all models and prioritising F1-score and ROC-AUC as primary evaluation metrics throughout the study.

### 5.2 Plot 2 — Default Rate by Payment Status (PAY\_0)



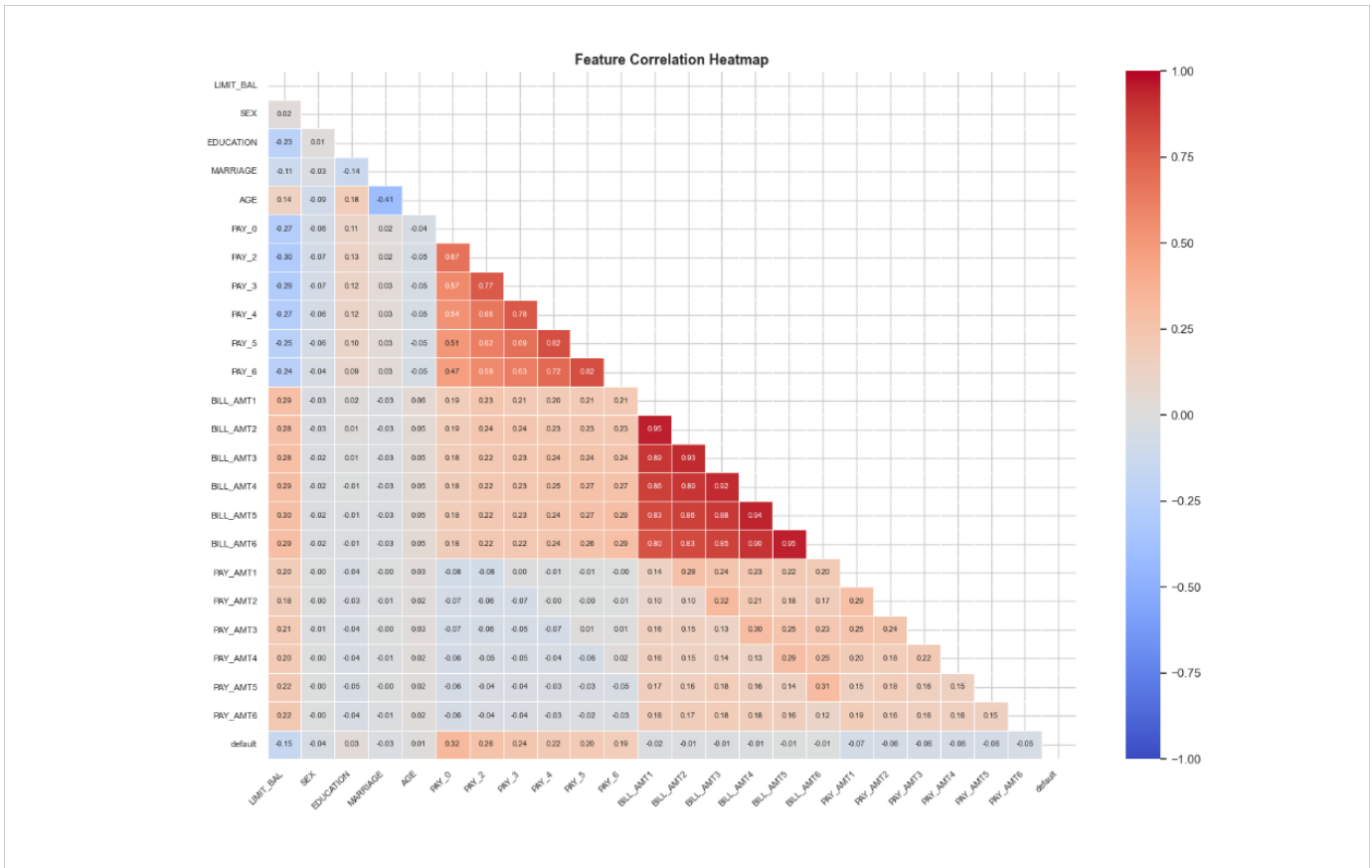
Default rate increases sharply with payment delay. Customers who paid on time (-1) show approximately 10% default rate, while those with two or more month delays exceed 60%. The steep jump between value 0 (revolving credit) and value 1 (one month late) indicates that any missed payment is a strong warning signal. PAY\_0 was confirmed as the strongest single predictor in the dataset through both this plot and the correlation heatmap.

**5.3 Plot 3 — Default Rate by Credit Limit (LIMIT\_BAL)**



Customers with credit limits of 0-50k NTD default at approximately 30%, while those with limits above 500k default at under 10%. The monotonically decreasing pattern confirms the hypothesis that credit limit functions as a proxy for the bank's own historical risk assessment. LIMIT\_BAL therefore carries both predictive signal about future behaviour and historical signal about the bank's prior evaluation.

## 5.4 Plot 4 — Feature Correlation Heatmap



The six BILL\_AMT columns form a high-correlation block (0.7-0.9), as expected from six consecutive monthly statements for the same account. PAY\_0 shows the strongest positive correlation with the target variable, consistent with Plot 2. This multicollinearity will not affect Decision Tree or kNN, but may inflate Logistic Regression coefficient estimates for the BILL\_AMT group

## Section 6: Modelling

---

Three classifiers were implemented representing different algorithmic families: a linear model (Logistic Regression), a tree-based model (Decision Tree), and an instance-based model (kNN). This diversity enables a meaningful comparison of decision boundary types, interpretability, and performance on this dataset.

### 6.1 Logistic Regression

Logistic Regression models the probability of default as a logistic function of a linear combination of input features. It is the standard baseline for binary credit risk classification in industry. Key hyperparameter choices:

- `class_weight="balanced"` :automatically compensates for the 22% class imbalance by weighting minority-class samples inversely proportional to their frequency.
- `max_iter=1000` :default of 100 iterations was insufficient for convergence on this 23-feature dataset.
- Solver: `lbfgs` (default) : efficient for small-to-medium tabular datasets with L2 regularisation.

### 6.2 Decision Tree

A Decision Tree partitions the feature space through binary splits, selecting the feature and threshold that maximise Gini purity at each node. An untuned baseline with no depth limit achieved ROC-AUC of 0.622 on the validation set:

| Parameter                | Values Tested   | Best Value | Effect                                   |
|--------------------------|-----------------|------------|------------------------------------------|
| <b>max_depth</b>         | 3, 5, 7, 10, 15 | 5          | Limits tree depth; prevents memorisation |
| <b>min_samples_split</b> | 2, 10, 20, 50   | 50         | Node must have 50+ samples to split      |
| <b>min_samples_leaf</b>  | 1, 5, 10        | 5          | Every leaf must contain 5+ samples       |

GridSearchCV used 5-fold cross-validation with `roc_auc` scoring across 60 parameter combinations (300 total fits). Best cross-validation ROC-AUC: 0.7604.



## 6.3 k-Nearest Neighbours (kNN)

kNN classifies a customer by finding the k most similar training examples and assigning the majority class. It has no `class_weight` parameter, so imbalance was addressed by tuning k. GridSearchCV was applied across 28 combinations (140 fits):

| Parameter          | Values Tested           | Best Value | Effect                                             |
|--------------------|-------------------------|------------|----------------------------------------------------|
| <b>n_neighbors</b> | 3, 5, 7, 11, 15, 21, 31 | 31         | Larger neighbourhood smooths majority-class bias   |
| <b>weights</b>     | uniform, distance       | uniform    | Equal weight to all neighbours                     |
| <b>metric</b>      | euclidean, manhattan    | manhattan  | L1 distance more robust to financial data outliers |

Best cross-validation ROC-AUC: 0.7546. The jump from k=5 to k=31 was the most impactful tuning decision — larger k reduces the majority-class dominance problem inherent in kNN on imbalanced datasets.

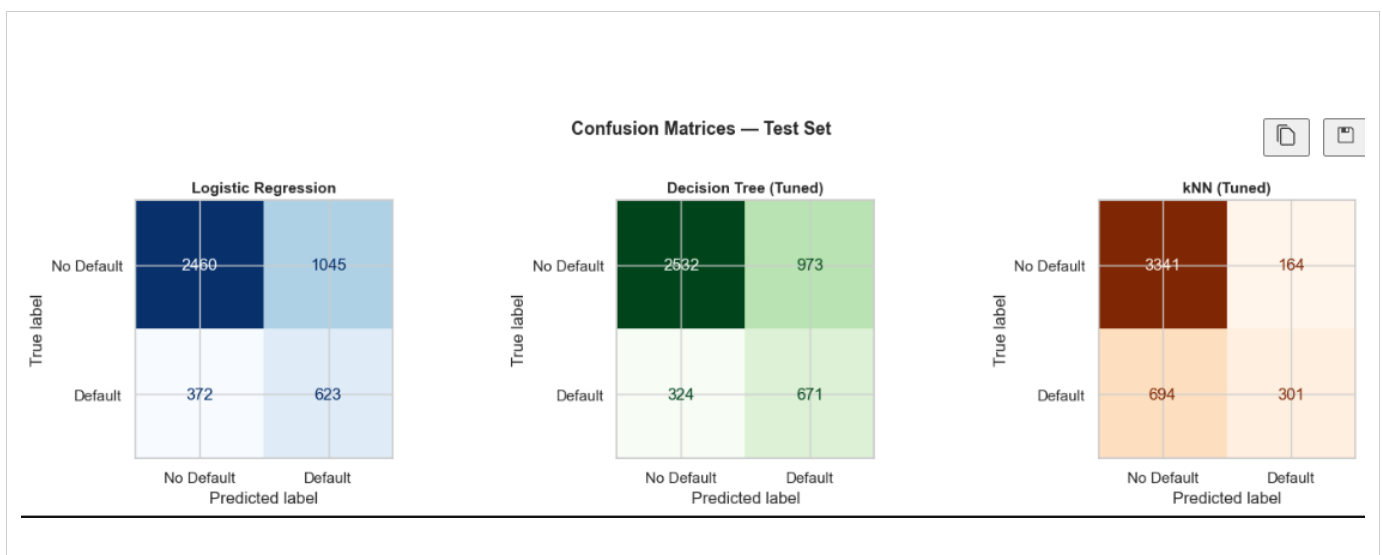
## Section 7: Performance Evaluation

All three models were evaluated on the held-out test set (4,500 rows), which was untouched throughout development. These are the final performance numbers for the study.

### 7.1 Final Test Set Results

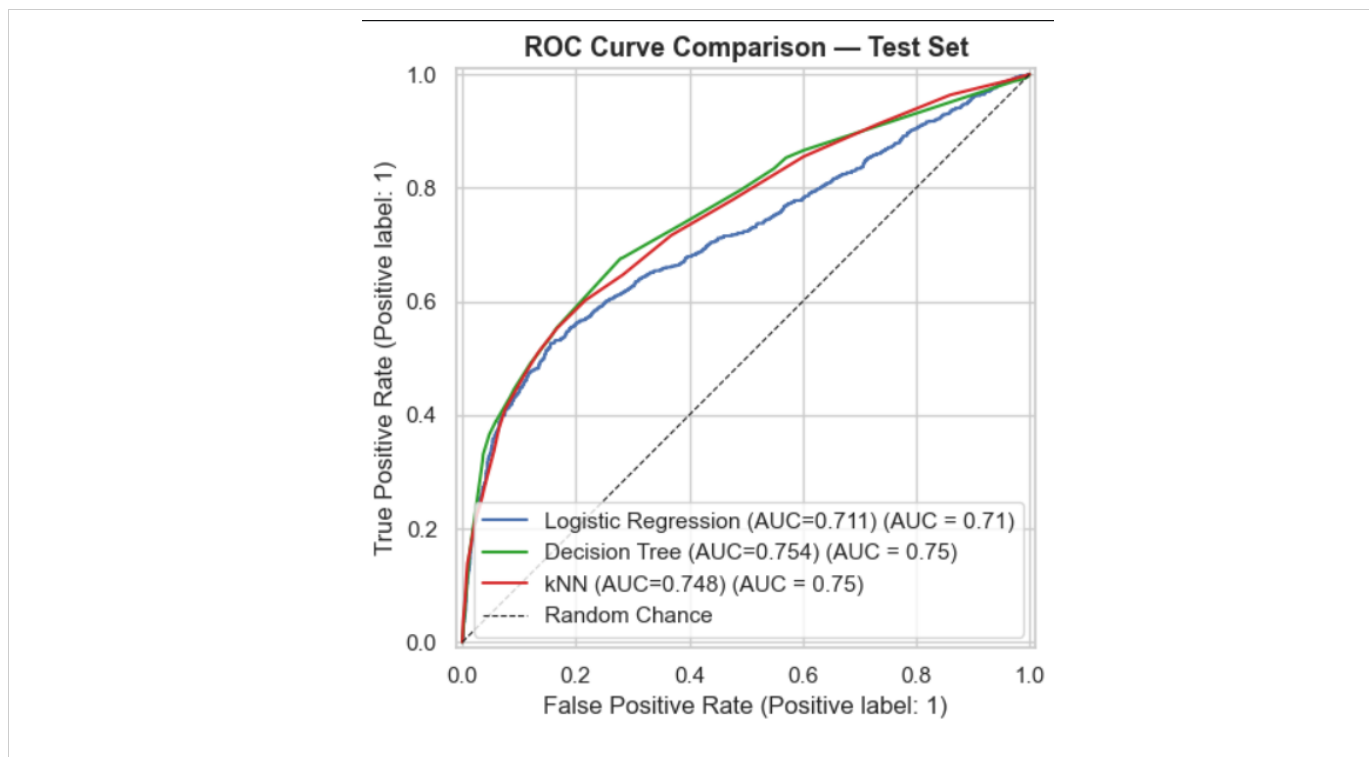
| Model                        | Accuracy | F1-Score | ROC-AUC |
|------------------------------|----------|----------|---------|
| <b>Logistic Regression</b>   | 0.6851   | 0.4679   | 0.7107  |
| <b>Decision Tree (Tuned)</b> | 0.7118   | 0.5085   | 0.7544  |
| <b>kNN (Tuned)</b>           | 0.8093   | 0.4123   | 0.7481  |

### 7.2 Confusion Matrices



| Model               | True Positives | False Negatives | False Positives | True Negatives |
|---------------------|----------------|-----------------|-----------------|----------------|
| Logistic Regression | ~623           | 372             | 1,045           | ~2,460         |
| Decision Tree       | 671            | 324             | 973             | ~2,532         |
| kNN                 | 301            | 694             | 164             | ~3,341         |

### 7.3 ROC Curves



The ROC curve plots True Positive Rate against False Positive Rate at every probability threshold. The Decision Tree curve sits above both other models across the full threshold range, confirming its superior discrimination ability. All three models significantly outperform the random-chance diagonal (AUC = 0.5).

## Section 8: Model Comparison

---

### 8.1 Results Summary

| Model                 | Accuracy | F1-Score | ROC-AUC | FN (missed defaults) | Verdict                        |
|-----------------------|----------|----------|---------|----------------------|--------------------------------|
| Logistic Regression   | 0.6851   | 0.4679   | 0.7107  | 372                  | Weakest on all metrics         |
| Decision Tree (Tuned) | 0.7118   | 0.5085   | 0.7544  | 324                  | Best overall — recommended     |
| kNN (Tuned)           | 0.8093   | 0.4123   | 0.7481  | 694                  | Misleading accuracy — worst FN |

### 8.2 Why Decision Tree is the Best Model

Decision Tree is the recommended model based on three converging lines of evidence:

- Highest F1-Score (0.5085): F1 penalises both false positives and false negatives. Decision Tree best balances catching defaulters (recall) with avoiding excessive false alarms (precision).
- Highest ROC-AUC (0.7544): The Decision Tree's ROC curve sits above both other models across all probability thresholds, confirming superior overall discrimination ability.
- Fewest missed defaults (FN = 324): The Decision Tree correctly identified 671 of 995 actual defaulters in the test set — more than twice the 301 caught by kNN. In banking, a missed defaulter represents a direct financial loss to the institution.

### 8.3 Why kNN's 80.9% Accuracy is Misleading

kNN achieved the highest raw accuracy (80.9%) but the lowest F1-Score (0.4123) and the highest false negative count (694). This apparent contradiction is explained by class imbalance. kNN missed 694 out of 995 actual defaulters, a 70% miss rate on the class that matters. Its high accuracy comes entirely from correctly classifying the 3,500+ non-defaulters, which requires near-zero predictive ability given the 78% majority. This is the classic accuracy paradox in imbalanced classification and is the reason F1-score and ROC-AUC are the appropriate primary metrics for this problem.

### 8.4 Why Logistic Regression Underperformed

Logistic Regression produced the weakest results on all three metrics. The most likely cause is its linear decision boundary :credit default behaviour involves complex interactions between features that a linear model cannot capture without explicit feature engineering. Additionally, the high multicollinearity among the six BILL\_AMT columns (0.7-0.9, identified in EDA) inflates the variance of corresponding coefficients, reducing the model's reliability.

## Section 9: Error Analysis

### 9.1 Misclassification Summary

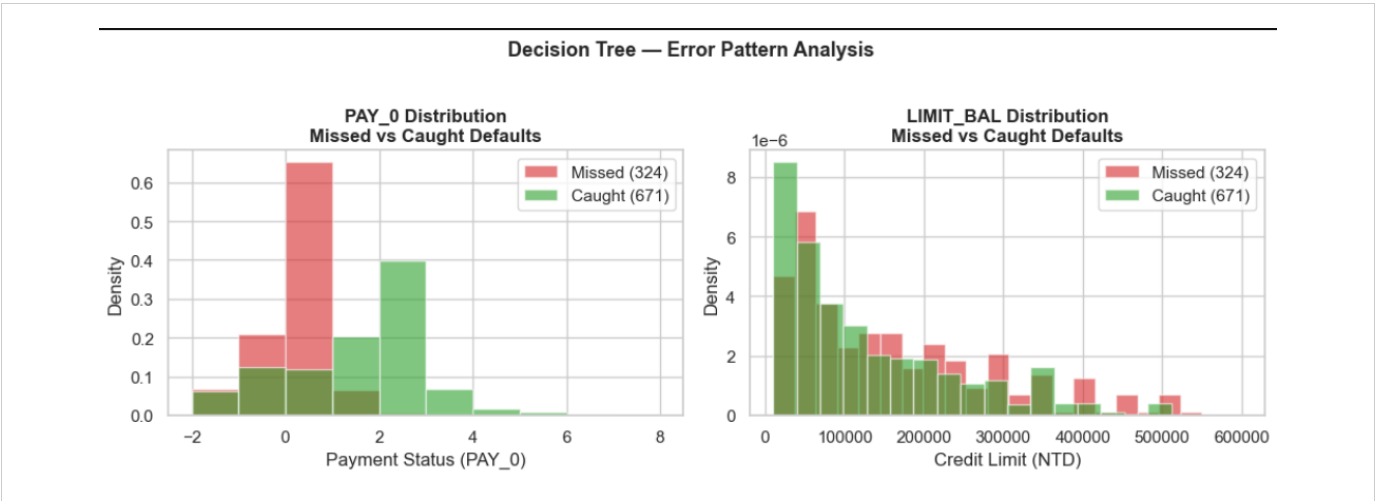
| Model                 | Total Errors | Error Rate | False Negatives       | False Positives      |
|-----------------------|--------------|------------|-----------------------|----------------------|
| Logistic Regression   | 1,417        | 31.5%      | 372 (missed defaults) | 1,045 (false alarms) |
| Decision Tree (Tuned) | 1,297        | 28.8%      | 324 (missed defaults) | 973 (false alarms)   |
| kNN (Tuned)           | 858          | 19.1%      | 694 (missed defaults) | 164 (false alarms)   |

The most important column is False Negatives. In a banking context, a false negative means a customer who will default is predicted as safe because the bank extends credit and incurs the full loss. kNN's low total error count is deeply misleading: 694 of its 858 errors are missed defaults, meaning it fails on the most costly error type at nearly twice the rate of Decision Tree.

### 9.2 Who Does the Decision Tree Miss?

Analysis of the 324 missed defaults versus the 671 correctly identified defaults for the Decision Tree reveals a clear pattern:

| Feature   | Missed Defaults (avg) | Caught Defaults (avg) | Interpretation                                       |
|-----------|-----------------------|-----------------------|------------------------------------------------------|
| PAY_0     | -0.3                  | 1.1                   | Missed customers paid on time recently — appear safe |
| LIMIT_BAL | 160,124 NTD           | 127,943 NTD           | Missed customers have higher credit limits           |
| BILL_AMT1 | 59,682 NTD            | 45,897 NTD            | Missed customers carry higher recent balances        |
| PAY_AMT1  | 5,015 NTD             | 2,460 NTD             | Missed customers made larger recent payments         |
| AGE       | 36.5 years            | 36.0 years            | No meaningful difference by age                      |



### 9.3 Error Pattern Interpretation

The Decision Tree misses defaulters who appear financially healthy in the most recent month: they have low PAY\_0 values (recent on-time payments), higher credit limits (suggesting the bank previously trusted them), and were making larger payments recently.

This failure pattern is consistent with a fundamental dataset limitation: the six-month window captures historical behaviour but cannot anticipate future disruptions.

### 9.4 Limitations

- Six-month window only: Longer payment history would enable detection of gradual financial deterioration patterns invisible in short windows.
- Class imbalance ceiling: Even with `class_weight="balanced"`, the 22% minority class limits how much the models can learn about default patterns.
- kNN production impracticality: kNN requires computing distances to all 21,012 training examples at prediction time. This is computationally impractical for a real bank processing millions of customers.
- BILL\_AMT multicollinearity: The high correlation among the six bill amount columns limits Logistic Regression coefficient interpretability.

## Section 10: AI Tool Usage & Reflection

---

AI tools were used primarily to clarify unfamiliar concepts and debug specific errors. All design decisions, model choices, and interpretations were made independently by the team. Every AI explanation was verified by running code and inspecting outputs.

### Example 1 — Understanding GridSearchCV

Prompt: "What is GridSearchCV and how does it work?"

The explanation of systematic hyperparameter search via cross-validation helped the team choose meaningful parameter ranges for Decision Tree tuning (e.g., `max_depth` values of 3, 5, 7, 10, 15 based on understanding of overfitting). The explanation was verified by observing the baseline-to-tuned AUC improvement ( $0.622 \rightarrow 0.754$ ), which confirmed the intuition about depth constraints.

What was wrong or misleading: Nothing conceptually incorrect, but the explanation alone was insufficient because the team had to select parameter ranges and interpret results independently.

### Example 2 — Debugging a Code Error

Error encountered: "ValueError: Found input variables with inconsistent numbers of samples: [25500, 30000]"

The bug was identified as using `stratify=y` (30,000 rows) in the second `train_test_split` call instead of `stratify=y_temp` (25,500 rows). The fix was a single parameter change. The fix was confirmed by checking split sizes: 21,012 / 4,488 / 4,500 summing to 30,000.

### Example 3 — Clarifying Metric Interpretation

Prompt: "kNN has the highest accuracy but lowest F1. Why is Decision Tree considered better here?"

The clarification of the accuracy paradox in imbalanced classification helped the team articulate why kNN's 80.9% accuracy is misleading. The explanation was verified directly from confusion matrix results: kNN's False Negative count of 694 versus Decision Tree's 324 confirmed that kNN fails on the most important error type despite its high accuracy.

## Conclusion

---

This study implemented and evaluated three supervised classification models on the Credit Card Default dataset (30,000 instances, 23 features, binary target). The preprocessing pipeline delivered a clean, correctly split, and scaled dataset with all design decisions explicitly justified. EDA revealed a 22.1% class imbalance, identified PAY\_0 as the strongest predictor, confirmed LIMIT\_BAL as a meaningful feature, and flagged multicollinearity among the BILL\_AMT columns.

Among the three classifiers, the tuned Decision Tree (max\_depth=5, min\_samples\_split=50, min\_samples\_leaf=5) is the recommended model, achieving the highest F1-Score (0.5085) and ROC-AUC (0.7544) on the held-out test set. It correctly identified 671 of 995 actual defaulters which is more than twice the 301 caught by kNN. Logistic Regression provided a meaningful linear baseline but was limited by its inability to capture non-linear feature interactions. kNN achieved the highest raw accuracy (80.9%) but the worst performance on the metrics that matter for imbalanced binary classification.

Error analysis revealed that all models struggle with a specific subgroup: customers who appear financially healthy in the most recent month (PAY\_0 near -1, higher credit limits, recent on-time payments) but default shortly after. This pattern requires longitudinal data beyond the six-month window available in this dataset and represents the primary opportunity for future improvement.